

第十一章 规划与智能体

规划旨在为目标任务制定包含一系列动作的解决方案，是大语言模型解决复杂问题能力的重要体现，也是自主智能体最重要的核心能力。自主智能体作为大语言模型的关键应用方向之一，被视为实现通用人工智能的极具潜力的技术路径。通过感知环境、规划解决方案以及执行相应动作，自主智能体能够有效完成既定目标任务。基于制定的任务的解决方案，自主智能体在环境中执行相应的动作，最终完成目标任务的求解。本章将首先介绍基于大语言模型的基本规划框架（第 11.1 节），主要包含方案生成与反馈获取两大关键步骤。在此基础上，我们将深入探讨如何构建基于大语言模型的智能体系统，并介绍其在不同场景下的应用（第 11.2 节）。

11.1 基于大语言模型的规划

虽然上下文学习和思维链提示方法形式上较为简洁且较为通用，但是在面对诸如几何数学求解、游戏、代码编程以及日常生活任务等复杂任务时仍然表现不佳 [298]。为了解决这类复杂任务，可以使用基于大语言模型的规划（Planning）。该方法的核心思想在于将复杂任务分解为若干相关联的子任务，并围绕这些子任务制定包含一系列执行动作（Action）的解决方案，从而将复杂任务的求解转换为一系列更为简单的子任务依次求解，进而简化了任务难度。本节将介绍基于大语言模型的规划方法，这也是后续大语言模型智能体（详见第 11.2 节）的技术基础。

11.1.1 整体框架

如图 11.1 所示，基于大语言模型的规划方法主要由三个组件构成，包括任务规划器（Task Planner）、规划执行器（Plan Executor）以及环境（Environment）¹。具体来说，大语言模型作为任务规划器，其主要职责是生成目标任务的解决方案。该方案包含一系列执行动作，每个动作通过合适的形式进行表达，例如自然语言描述或代码片段。对于长期任务，任务规划器还可以引入存储机制，用于解决方

¹ 尽管这个范式在结构上与强化学习有些相似，但是我们将规划和执行过程进行了显式的解耦，这与强化学习中通常将两者耦合在智能体中的做法有所不同。这个范式的定义相对宽泛，主要是为了帮助读者深入理解规划方法的基本思想。

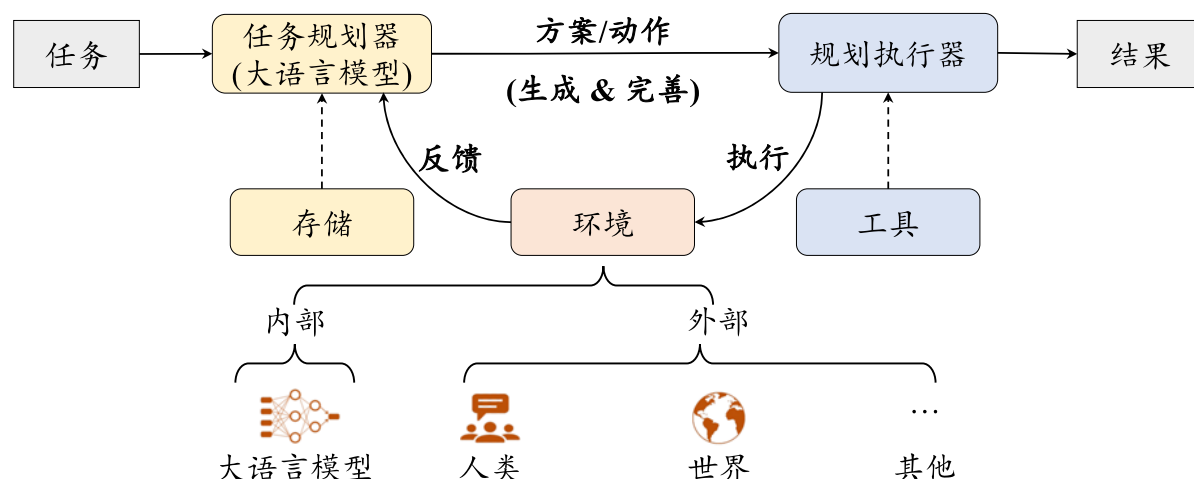


图 11.1 大语言模型通过基于提示的规划解决复杂任务的流程（图片来源：[10]）

案与中间执行结果的存储与检索。规划执行器则负责执行解决方案中所涉及到的动作。根据任务性质的不同，规划执行器可以由大语言模型实现，也可以由执行具体物理任务的实体（如机器人）来实现。环境是规划执行器实施动作的具体场景，不同任务对应着不同的执行环境，例如 Web 互联网或像 Minecraft 这样的外部虚拟世界。

在解决复杂任务时，任务规划器首先规划解决方案，既可以一次性生成包含所有子步骤的详尽动作序列，也可以迭代地生成下一步骤所需执行的动作（详见第 11.1.2 节）。然后，规划执行器在执行环境中执行解决方案中所涉及到的动作，并由环境向任务规划器提供反馈信息（详见第 11.1.3 节）。任务规划器可以进一步利用这些反馈信息来优化或继续推进当前的解决方案，并通过迭代上述过程完善任务解决方案。下面将详细介绍这两个关键步骤。

11.1.2 方案生成

方案生成主要是基于大语言模型的综合理解与推理能力，通过合适的提示让大语言模型生成目标任务的解决方案。一般来说，解决方案（或者其中包含的中间步骤）可以采用自然语言表达或者代码表达的形式。自然语言的形式较为直观，但由于自然语言的多样性与局限性，不能保证动作被完全正确执行，而代码形式则较为严谨规范，可以使用外部工具如代码解释器等保证动作被正确执行。图 11.2 对比展示了采用自然语言表达和代码表达的执行方案。在现有的研究中，任务规划器主要采用两种规划方法：一次性的方案生成和迭代式的方案生成。具体来说，一次性方案生成方法要求任务规划器直接生成完整的解决方案（包含所有子步骤

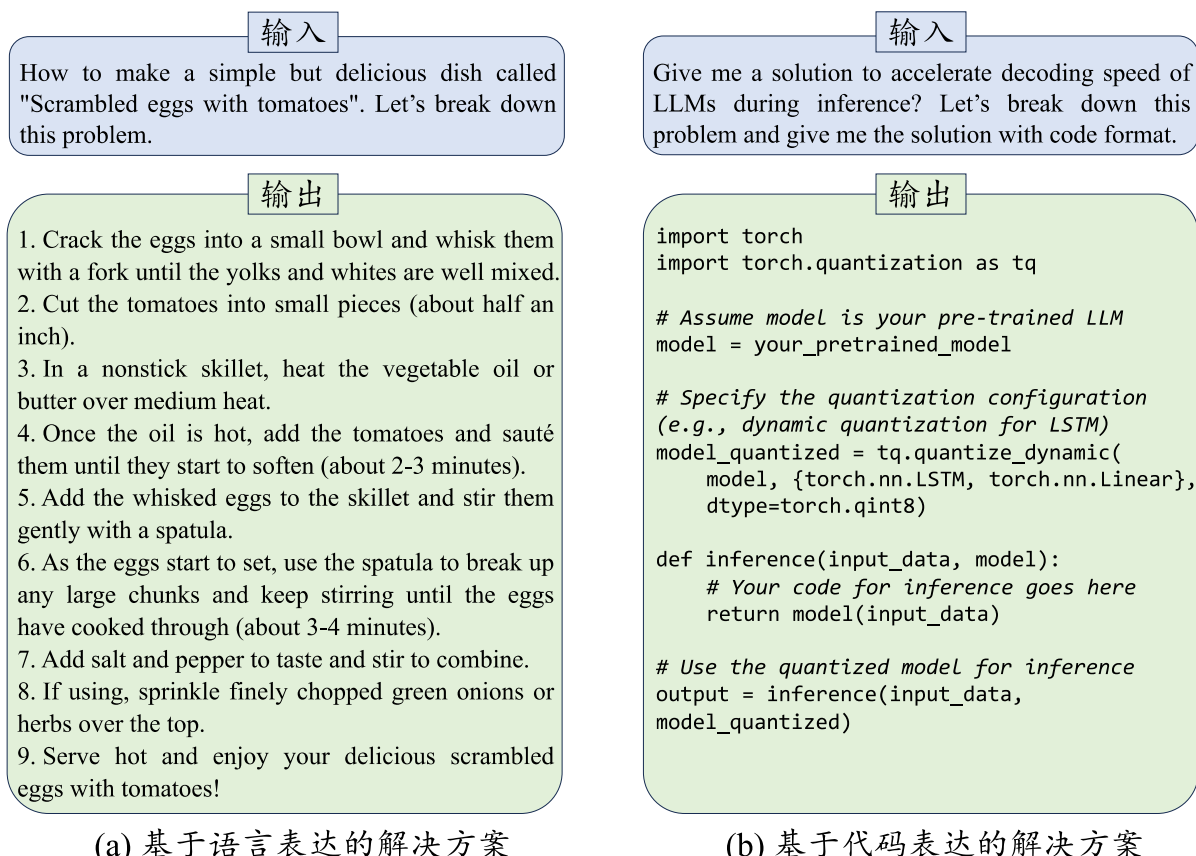


图 11.2 基于语言表达和基于代码表达的方案生成示例对比

的详尽动作序列), 这一方法实现较为简单, 但是容错性较低, 一旦中间某个步骤出错, 就容易导致最终执行结果出错。而迭代式方案生成方法则通过与环境进行交互, 逐步地生成下一步动作计划, 能够根据环境反馈对中间执行步骤进行修正与调整。下面具体介绍一次性方案生成和迭代式方案生成这两类方法。

一次性方案生成

这种方法通过特殊设计的提示方法让大语言模型一次性生成完整的解决方案, 生成的方案通常包含一系列供规划执行器执行的动作描述。例如, 可以在提示中加入如 “*Let's break down this problem.*” 这样的指令, 并通过上下文学习的方式提示大语言模型生成目标任务的解决方案 [299]。图 11.2 (a) 展示了一个提示询问大语言模型如何制作番茄炒蛋的例子。可以看到, 针对这个问题, 大语言模型生成了对应的解决方案, 并将执行步骤序列表达为自然语言文本形式。此外, 还可以使用代码表达的形式来表示具体的执行方案。如图 11.2 (b) 所示, 输入一个编程问题及对应的提示给大语言模型, 大语言模型生成了一个基于代码表达的解决方案。

在实际应用时, 需要根据任务特性来选择具体的规划方案形式。一般来说, 如

果待解决任务需要较强的推理逻辑或数值计算能力，则推荐使用基于代码表达的方案生成。如果待解决任务的形式不固定、难以进行形式化表达，如多跳问答、信息检索或推荐任务，则推荐使用基于自然语言的表达。这一建议对于下面介绍的迭代式动作生成方法同样适用。

迭代式方案生成

在这一类方法中，大语言模型基于历史动作和当前环境的反馈逐步规划下一步的执行动作。一个具有代表性的方法是 ReAct [300]，其核心动机是在让大语言模型在规划动作时模拟人类“先思考-再决策”的行为方式。具体来说，该方法首先通过提示让大语言模型思考当前状态下应该采取何种决策，并生成决策理由与相应的执行动作。然后，规划执行器在外部环境中执行动作并将交互信息反馈给任务规划器，然后由任务规划器基于反馈信息生成下一步的决策理由与执行动作。任务规划器通过迭代上述过程，逐步思考并生成新的动作，直至解决任务。但是，在上述过程中，如果某一步动作不是最优解或出现错误，任务规划器也只能继续向前规划直至结束，最终可能导致整个方案只能获得次优结果甚至失败。为了缓解这一问题，可以使用回溯策略（Back-tracing）让任务规划器回退到上一步所对应的状态，从而通过探索其他执行动作来优化最终的解决方案。思维树就采用了类似的回溯策略 [289]（参考第 10.3.2 节）。

图 11.3 展示了 ReAct 解决多跳问答任务的提示设计和完整规划过程。具体来说，在每一步动作规划时，任务规划器基于历史动作及其反馈生成下一步的决策思考和相应的动作，然后规划执行器执行下一步动作并从环境中获得相应的反馈，重复上述过程直至解决问题。例如，在第二步动作规划时，任务规划器基于“动作 1”（搜索“Grown-Ups”的相关信息）和“反馈 1”（“Grown-Ups”的相关信息）生成了下一步的“思考 2”（理解“Grown-Ups”的相关信息）和“动作 2”（搜索“Allo Allo!”的相关信息），然后规划执行器执行“动作 2”，并获得“反馈 2”（“Allo Allo!”的相关信息）。

（首先，输入问题和一些相关的问题-解决方案的示例（此处为方便展示，我们省略示例的具体内容），任务规划器进行第一轮规划。首先，生成第一步动作，其内容具体包括“思考 1”和“动作 1”。）

第一轮规划： Question: Grown-Ups starred the actor who was best known for which role on “Allo Allo!”?

思考 1: I need to search Grown-Ups and “Allo Allo!”, find the actor who starred in Grown-Ups, then find the role he was best known for in “Allo Allo!”.

动作 1: Search[Grown-Ups]

（然后，规划执行器执行“动作 1”，并将得到反馈，即“反馈 1”。）

反馈 1: Grown-Ups is a 1980 British BBC television film devised and ... on 28 November 1980.

（在后续步骤中，将历史涉及的动作和反馈拼接在一些输入给任务规划器，然后继续生成下一步的动作，以此类推，直至第四步，生成“动作 4”，即停止推理。）

思考 2: Grown-Ups starred Lesley Manville, Philip Davis, Brenda Blethyn, Janine Duvitski, Lindsay Duncan and Sam Kelly. I need to search “Allo Allo!” and find which actor was best known for which role.

动作 2: Search[“Allo Allo!”]

反馈 2: Could not find [“Allo Allo!”] ... on 28 November 1980.

...

思考 4: Gorden Kaye was best known for playing womanising café owner René Artois in the television comedy series “Allo Allo!”. So the answer is Rene Artois.

动作 4: Finish[Rene Artois]

（执行“动作 4”后，得到最终结果，即“反馈 4”。）

反馈 4: Answer is INCORRECT

（执行后发现答案不正确。然后，我们将完整解决方案及每一步动作对应的反馈拼接输入给任务规划器，同时添加一些相关的问题-解决方案-反思的示例（此处为方便展示，我们省略示例具体内容）帮助任务规划器生成对当前解决方案的反思，即“反思 1”。）

反思 1: I searched the wrong title for the show, “Allo Allo!”, which resulted in no results. I should have searched the show’s main character, Gorden Kaye, to find the role he was best known for in the show.

（上述是一轮完整的规划，我们将上一轮不正确的完整求解策略及其反馈，以及反思拼接输入给任务规划器，其重新生成新的求解策略的每一步，以此类推，直至得到成功的反馈。）

第二轮规划： Question: Grown-Ups starred the actor who was best known for which role on “Allo Allo!”?

思考 1: I need to search Grown-Ups ... in “Allo Allo!”.

动作 1: Search[Grown-Ups]

反馈 1: Grown-Ups is a 1980 British BBC television film devised and ... on 28 November 1980.

（可以看到，在加入“反思 1”后，模型在第二步生成时，对“思考 2”和“动作 2”进行了修正，我们用下滑线进行了强调。）

思考 2: The paragraph does not mention the actor who was best known for a role on “Allo Allo!”. I need to search the actor’s name instead.

动作 2: Search[Sam Kelly]

反馈 2: Roger Michael Kelly... Sam in On the Up, and Ted Liversidge in Barbara.

思考 3: Sam Kelly is best known for his role as Captain Hans Geering in “Allo Allo!”, so the answer is Captain Hans Geering.

动作 3: Finish[Captain Hans Geering]

反馈 3: Answer is CORRECT

（执行后答案正确，结束规划求解。）

图 11.3 使用 ReAct 方法（单轮规划）与 Reflexion 方法（多轮规划）求解多跳问答任务示例

11.1.3 反馈获取

在任务规划器生成完整的解决方案或下一步动作后，规划执行器在环境中执行对应的动作。在执行动作后，规划执行器会将环境的反馈信号传递任务规划器。这些反馈信号可以用于完善整体解决方案或规划下一步动作。根据任务规划器与环境之间的交互方式，环境反馈可以分为两类，包括外部反馈和内部反馈，下面进行具体介绍。

外部反馈

外部对象可以为任务规划器提供重要的反馈信号。这里以物理工具、人类以及虚拟环境这三种外部对象为例，对他们所提供的反馈信号进行介绍。首先，物理工具，如代码解释器等，在编程或数学任务的解决过程中起到了关键的作用。它们可以直接执行基于代码形式的解决方案或动作，并能够将执行结果反馈给任务规划器，帮助其进行规划改进。例如，当代码执行出现错误时，解释器会迅速将错误信息反馈给任务规划器，使其能够及时修正后续的方案生成。其次，在具身智能场景中，人类成为了任务规划器获取反馈的重要来源。当机器人在物理世界中与人类进行交互时，人类能够根据机器人的询问或动作，提供关于物理世界的实时信息。这些信息对于任务规划器来说至关重要，因为它们能够帮助机器人更好地感知和理解周围的环境。例如，当机器人执行动作“走到抽屉前并询问当前抽屉是否打开？”，人类可以反馈抽屉的实时状态，帮助任务规划器更好地感知和理解物理世界。最后，在游戏领域，虚拟环境能够为任务规划器提供实时的动作执行反馈，从而协助其更加高效地完成后续的游戏任务。

内部反馈

除了外部反馈，大语言模型本身也能够对任务规划器提供反馈信息。首先，大语言模型可以直接判断当前动作是否规划正确。具体来说，可以将历史动作序列以及对应的反馈输入给大语言模型，通过使用类似 *“Is the current action step being taken correct or not?”* 的指令，让大语言模型检查当前动作的正确性，并给出反馈结果。在得到完整的解决方案后，规划执行器可以在环境中执行该方案，并且获得相应的外部反馈信息。通常来说，这些外部反馈所传达的信息相对有限，例如只是简单地示意了执行结果错误或异常等。

为了更好地理解执行结果的背后原因，大语言模型可以将简单的环境反馈（例如成功或失败）转换为信息量更为丰富的、自然语言表达的总结反思，帮助任务

规划器重新生成改进的解决方案。一个代表性的工作是 Reflexion [301]，该方法旨在借助大语言模型的分析与推理能力，对于当前方案的执行结果给出具体的反思结果，用于改进已有的解决方案。在实现中，需要在提示里包含已执行的任务方案及其环境反馈，还可能需要引入相关的上下文示例。例 11.3 展示了 Reflexion 方法在多跳问答任务中的应用。具体来说，首先将上一轮解决方案及其反馈输入给任务规划器生成反思（如“反思 1”）。然后，将上一轮解决方案、反馈和反思输入给任务规划器，重新生成新一轮的解决方案，以此类推，直至得到成功的反馈。可以看到，在加入“反思 1”后，模型在第二轮第二步动作规划时，将第一轮第二步动作（搜索“Allo Allo!”的相关信息）修正为新的“动作 2”（搜索“Sam Kelly”的相关信息）。

11.2 基于大语言模型的智能体

智能体（Agent）是一个具备环境感知、决策制定及动作执行能力的自主算法系统 [302]。研发智能体的初衷在于模拟人类或其他生物的智能行为，旨在自动化地解决问题或执行任务。然而，传统智能体技术面临的主要挑战是它们通常依赖于启发式规则或受限于特定环境约束，很大程度上限制了它们在开放和动态场景中的适应性与扩展性 [303]。由于大语言模型在解决复杂任务方面展现出来了非常优秀的能力，越来越多的研究工作开始探索将大语言模型作为智能体的核心组件，以提高智能体在开放领域和动态环境中的性能 [304]。本节将首先简要回顾智能体的发展历程，然后详细介绍基于大语言模型的智能体框架，最后讨论智能体在各种应用场景中的潜在用途和面临的挑战。

11.2.1 智能体概述

在人工智能发展的早期阶段，基于规则的方法占据了智能体技术的主导地位 [305]。通过专家预先定义好的规则和逻辑，这些智能体能够在一些特定任务上模拟人类的决策过程，进而完成相应任务。但受限于预定义的规则和知识库，早期的智能体往往表现出较低的适应性和灵活性，无法有效应对未经历过的应用场景。随着机器学习（特别是深度学习）技术的兴起，基于模型的智能体开始受到了广泛关注。这类智能体不再依赖于预先定义的规则，而是基于环境中的特征来构建可学习的决策模型。在基于模型的智能体中，强化学习方法扮演了重要角色。

强化学习智能体通过与环境的交互来学习最佳行为策略 [306]。它们通过探索和利用，不断进行试错并根据所获得的环境反馈信息来调整自己的行为，从而最大化累积奖励。这种方法在游戏、自动驾驶等领域取得了显著成果。最近，大语言模型得到迅速发展，其具有强大的学习和规划能力，能够处理更加复杂、抽象的任务，并在自然语言理解、图像识别、推理决策等方面展现出前所未有的性能。基于大语言模型的智能体能够利用大语言模型的强大能力，从而自主、通用地与环境进行交互，成为了当前研究的热点 [304]。为了讨论的方便，在后续的内容中，我们简称“基于大语言模型的智能体”为“大语言模型智能体”。

11.2.2 大语言模型智能体的构建

在本节中，我们介绍大语言模型智能体的构建过程，将围绕三个基本组件进行介绍，包括记忆组件 (Memory)、规划组件 (Planning)² 和执行组件 (Execution)。通过这些组件共同协作，智能体能够有效地感知环境、制定决策并执行规划的动作，进而完成相应任务。此外，本节将以推荐系统智能体框架 RecAgent 为例 [114]，详细介绍大语言模型智能体各个组件，并基于 RecAgent 的用户模拟框架给出一个应用实例（如例 11.1 所示）。

记忆组件

人类的记忆系统是一种复杂而高效的信息处理系统，它能够储存新知识，并在需要时回顾和使用已存储的信息，以协助应对当前环境并做出明智的决策。类似地，在人工智能系统中，记忆组件构成了智能体的核心存储单元，主要用于存储智能体与环境的历史交互记录，并能够随时检索使用，这些信息可以是文本形式，也可以是图像、声音等多模态形式。例如，聊天机器人利用记忆组件来存储用户的偏好，进而提供更具个性化的服务体验。大语言模型智能体通过特殊设计的读写操作，将相关信息分别存储在短期记忆和长期记忆中，面对不同类型的需求时，智能体能够灵活地调用长短期记忆，以支持其复杂的认知与推理过程。

- **短期记忆**. 短期记忆是负责暂时存储和处理智能体相关信息的记忆载体。在大语言模型智能体中，短期记忆通常对应于模型内部的上下文窗口（即输入窗口），大语言模型通过推理等机制对于这些上下文信息进行读取操作。短期记忆中的信息存储持续时间相对较短，并且对于信息容量有一定的限制。大部分的短期记忆

²关于规划的详细内容，请参见第 11.1 节。本节视为智能体的一个组件进行介绍。

调用记忆组件：

长期记忆：

Name: Bob (gender: male; age: 25; traits: compassionate, caring, ambitious, optimistic; career: photographer; interest: sci-fi movies, comedy movies; feature: watcher, critic, poster).

Bob recently heard ['The Matrix', 'Back to the Future.', 'Anchorman', 'Superbad'] on social media.

Alice recently watched nothing on recommender system. Other than that Alice doesn't know any movies.

短期记忆：

It is September 12, 2023, 08:00 AM.

Most recent observations: Bob and Alice had a conversation about their shared interest in movies, discussing their favorite genres such as...

加入新记忆时对记忆重要性打分：

Observation: Alice want to post for all acquaintances.

Rating: 6

调用规划组件：

Plan: Bob first wants to enter the recommendation system to find a movie he is interested in, then watch the movie, and afterwards, chat with friends about the movie.

调用行动组件：

选择进入推荐系统或者社交平台：

[RECOMMENDER]: Bob enters the Recommender System

接受推荐，观看电影：

[RECOMMENDER]: Bob watches the movie <Interstellar>

智能体之间聊天：

[Bob]: Hey Alice! How's it going? I heard you were interested in a movie. What's been on your mind?

[Alice]: Hey Bob! I'm doing great, thanks for asking. Yeah, I've been hearing a lot about this movie <Interstellar> recently. Have you heard of it too?

[Bob]: Absolutely! Actually, I've been seeing it all over social media as well.

[Alice]: That's awesome! I'm glad you enjoyed it. I've been wanting to watch it too. Would you be up for a cozy movie night to watch it together? We can discuss our thoughts and interpretations afterwards.

[Bob]: I'd love that! It's always more fun to watch movies with friends and have those deep conversations afterwards. Count me in!

...

例 11.1 推荐系统智能体 RecAgent 应用示例

只会使用一次，在必要时，短期记忆的内容可以转变为长期记忆存储。例 11.1 展示了 RecAgent 中短期记忆调用的操作示例。在这个例子中，大语言模型通过调取近期的观察，获取了用户在当前环境中的状态，将其作为短期记忆存储，具体包括当前时间和刚刚发生的事件。在下次交互中，模型会使用到这些历史信息，以便更准确地进行未来行动规划或行动执行。

- **长期记忆.** 长期记忆是智能体存储长期累积信息的记忆载体。长期记忆单元中的存储内容具有持久性，即使在不常访问的情况下也能稳定保留，涵盖事实知识、基础概念、过往经验以及重要技能等多个层面的信息。长期记忆的存储方式比较灵活，可以是文本文件、结构化数据库等形式，通常使用外部存储来实现。大语言模型通过检索机制读取长期记忆中的信息，并借助反思机制进行信息的写入与更新 [301]。当存储记忆的介质接近容量上限或出现重复记忆时，系统会及时启动清理机制，确保记忆的高效存储和利用。一般来说，智能体的角色和功能定义往往通过长期记忆来存储，这些重要信息通常存储在智能体的配置文件中 [304]。例 11.1 展示了 RecAgent 的长期记忆组件。在这个例子中，智能体的长期记忆包括用户的配置文件、近期的经历、对他人的观察、以及自身目前的状态。在后期进行规划和行动时，智能体会调用长期记忆中的相关记忆提供支持。

规划组件

规划组件为智能体引入了类似于人类解决任务的思考方式，将复杂任务分解为一系列简单的子任务，进而逐一进行解决。这种方法降低了一次性解决任务的难度，有助于提高问题解决的效率和效果，提高了智能体对复杂环境的适应性和操作的可靠性。对大语言模型智能体而言，可以采用多种规划形式，例如文本指令或代码程序。为了生成有效的规划方案，大语言模型智能体可以同时生成多个候选方案，并从中选择一个最佳方案用于执行。在应对复杂任务时，智能体还可以根据环境的实时反馈信息进行迭代优化改进，从而更高效地解决涉及复杂推理的问题。例 11.1 展示了智能体在任务开始时根据长短期记忆和环境制定初步规划方案，并在每一步行动前根据新接收到的信息对于当前规划方案进行细致调整，确保其行为的合理性。

执行组件

执行组件在智能体系统中承担了关键作用，它的主要职责是执行由规划组件制定的任务解决方案。通过设置执行组件，智能体可以产生具体的动作行为，进而与环境进行交互，并获得实际的执行效果反馈。执行组件的运作通常需要记忆

组件和规划组件进行协同。具体来说，智能体会在行动决策过程中执行规划组件制定的明确行动规划，同时会参考记忆组件中的长短期记忆来帮助执行准确的行动。在技术实现上，执行组件可以通过语言模型自身来完成预定规划 [307]，或者通过集成外部工具来增强其执行能力 [300]。例 11.1 展示了一个智能体如何根据记忆和既定规划来执行具体行为的过程。其中，智能体根据计划，首先进入了推荐系统，然后被推荐系统推荐了电影《Interstellar》并且进行观看，最后和其他智能体针对该电影进行了交流，完成了规划中的制定的系列动作。

工作流程

基于上述三个核心组件，下面系统地介绍大语言模型智能体在环境中的工作流程。这一流程通常遵循以下步骤：首先，智能体对当前状态进行理解和分析。在这一过程中，它可能会从记忆组件中检索相关的历史信息或知识，以便更全面地理解和分析当前状态。接下来，规划组件通过综合考虑长短期记忆组件中已存储的信息，生成下一个行动策略或计划。这一步骤涉及对多个执行方案进行预测与评估，以选择最优的行动路径。随后，执行组件负责根据规划组件生成的任务解决方案执行实际行动，并与当前环境产生交互。在执行过程中，智能体可能会借助外部工具或资源来增强自身的执行能力。最后，智能体通过感知单元或系统接口从环境中接收反馈信息，并将这些信息暂时存储于短期记忆中。智能体会对短期记忆中的新获取到的信息进行处理，例如舍弃掉和未来规划无关的观察。上述流程将作为新的记忆被记录在记忆组件中。

当接收到用户请求或面临特定任务时，智能体会按照这一既定流程与环境进行多轮交互，以逐步实现设定的任务目标。在这一过程中，大语言模型智能体还能够根据环境的实时反馈来动态调整自身的行为策略。例 11.1 展示了一个基于大语言模型智能体的推荐系统仿真示例。其中，一个智能体仿真了虚拟用户 Bob 在推荐系统中的交互行为，其先在虚拟环境中调用了自身的长短期记忆，包括朋友近期的观影行为与其自身的当前状态，然后制定了与电影相关的动作规划，并进行了一系列行动（在推荐系统中获取电影推荐、观影、与朋友交流）。

11.2.3 多智能体系统的构建

与单智能体系统的独立工作模式不同，多智能体系统着重强调智能体间的协同合作，以发挥集体智慧的优势。在多智能体系统中，可以从相同或不同类型的大语言模型中实例化出多个智能体，每个智能体均扮演特定角色并承担着对应功

能。通过智能体间的交互与协作，智能体系统的灵活性和适应性得到显著增强，能够完成相较于单智能体而言更为复杂、具有挑战性的任务。为了构建多智能体系统，智能体不仅需要具备自主性和决策能力，同时应能理解和预测其他智能体的行为，以及向其他智能体传递信息。因此，在设计和实现多智能体系统时，需要特别重视智能体间的交互机制和协作策略。本节将首先概述多智能体系统的构建方法，然后针对多智能体系统的通讯协同机制进行详细介绍。

多智能体系统的构建方法

要构建多智能体系统，首先需要明确多智能体系统整体需要解决的问题或实现的目标，可以针对特定任务，也可以针对某一个环境进行仿真模拟。随后，在系统内创建多个智能体实例（具体方法参阅第 11.2.2 节的介绍）。在创建智能体实例的过程中，需要根据问题的复杂性和所需功能，设计智能体的类型、数量和特性。例如，智能体的主要任务可以被设定为进行规划设计、获取新的知识、对事物或现象进行评判等类型，具体取决于应用场景以及任务目标。每个智能体应具备独特的功能特征、结构层次以及行为策略。

作为最为关键的一个步骤，接下来需要定义多智能体之间的交互方式，包括协作、竞争、信息交流等方面，以及制定协议、策略或博弈论规则，以确保智能体之间能够有效进行协同运作（下一小节将介绍多智能体系统的通讯协同机制）。此外，在构建多智能体系统的过程中，还需要考虑一些关键因素，如系统的可扩展性、可维护性、安全性以及智能体之间的异构性等。这些因素对于确保系统的长期稳定运行和持续改进至关重要。完成以上步骤后，一个多智能体系统就初步搭建完毕，接下来可以在具体应用环境中部署，并进行持续的监控和维护。

多智能体系统的通讯协同机制

在多智能体系统中，通讯机制与协同机制是实现智能体之间有效协作的重要基础技术。这两种机制的核心在于加强智能体之间的信息交流与能力共享。每个智能体都拥有自身的感知、学习和决策能力，但它们所能获取的信息和任务执行能力通常是有限的。通过通讯协同机制，可以实现智能体之间的信息共享、任务分配和协同控制，从而提高整个系统的性能和效率。下面介绍这两种重要机制。

- **通讯机制**. 多智能体系统的通讯机制通常包括三个基本要素：通讯协议、通讯拓扑和通讯内容。通讯协议规定了智能体之间如何进行信息交换和共享，包括通讯的方式、频率、时序等；通讯拓扑则定义了智能体之间的连接关系，即哪些智能体之间可以进行直接通讯，哪些需要通过其他智能体进行间接通讯；通讯内容

则是指智能体之间实际传输的信息，包括状态信息、控制指令、任务目标等，其形式可以是自然语言、结构化数据或者代码等。

- **协同机制**：多智能体系统的协同机制通常包括协作、竞争和协商。协作指的是智能体通过共享资源、信息和任务分配来实现共同目标；竞争则涉及到在资源有限的环境中，智能体之间的竞争关系，通过博弈论和竞价机制，使得整体系统可以在竞争中寻求最优解决方案；协商是指智能体通过交换信息和让步来解决目标或资源的冲突。

在实际应用中，多智能体系统的通讯与协同机制需要满足一定的要求，如实时性、可靠性、安全性等。实时性要求智能体之间的信息传输必须及时，以保证协同行为的实时响应；可靠性要求通讯系统必须稳定可靠，避免信息丢失或误传；安全性则要求通讯内容要受到保护，防止被恶意攻击或窃取。

11.2.4 大语言模型智能体的典型应用

大语言模型智能体在自主解决复杂任务方面展现出了巨大的潜力，不仅能够胜任特定任务，还可以构建面向复杂场景的虚拟仿真环境。本节将介绍三个大语言模型智能体的典型应用案例。

WebGPT

WebGPT [31] 是由 OpenAI 开发的一款具有信息检索能力的大语言模型，它基于 GPT-3 模型微调得到，可以看作是大语言模型智能体的一个早期雏形。WebGPT 部署在一个基于文本的网页浏览环境，用以增强大语言模型对于外部知识的获取能力。作为一个单智能体系统，WebGPT 具备自主搜索、自然语言交互以及信息整合分析等特点，能够理解用户的自然语言查询，自动在互联网上搜索相关网页。根据搜索结果，WebGPT 能够点击、浏览、收藏相关网页信息，对搜索结果进行分析和整合，最终以自然语言的形式提供准确全面的回答，并提供参考文献。WebGPT 在基于人类评估的问答任务中，获得了与真实用户答案准确率相当的效果。

MetaGPT

MetaGPT [308] 是一个基于多智能体系统的协作框架，旨在模仿人类组织的运作方式，模拟软件开发过程中的不同角色和协作。相关角色包括产品经理、架构师、项目经理、软件工程师及测试工程师等，并遵循标准化的软件工程运作流程对不同角色进行协调，覆盖了需求分析、需求文档撰写、系统设计、工作分配、

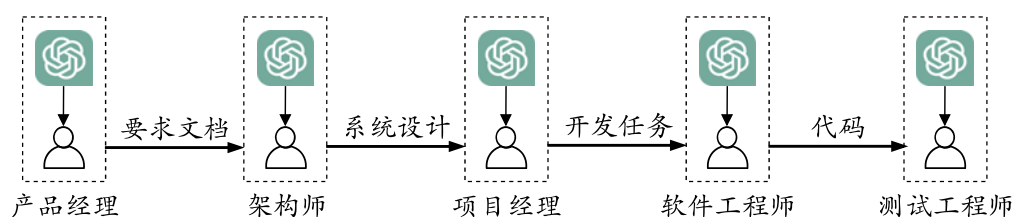


图 11.4 MetaGPT 执行软件开发工作的全流程示例

代码实现、系统测试等软件开发全生命周期，最终满足特定软件开发项目的需求。图 11.4 展示了 MetaGPT 运行的实际流程。MetaGPT 的框架分为基础组件层和协作层。基础组件层构建了支撑该多智能体系统的核心组件，包括环境、记忆、角色、行动和工具。进一步，协作层在基础组件层之上，定义了具体的通信协作机制，包括知识共享和封装工作流程等。与人类项目团队相比，MetaGPT 具有较高的开发效率与较低的投入成本，但是最终产出的代码并不都能保证成功运行，产出代码的执行成功率仍有待于进一步提升。

《西部世界》沙盒模拟

为了探索大语言模型智能体在社会模拟中的应用，研究人员于 2023 年提出了“生成式智能体”（Generative Agent）这一创新概念 [309]，并构建了类似《西部世界》的沙盒模拟环境。其中，多个智能体根据各自独特的人物背景（以自然语言形式描述人物身份的配置文件）在小镇中生活。这些模拟人物不仅能与其他人物进行自主交流，还能与环境进行丰富多样的交互，例如在图书馆看书、在酒吧喝酒，这些行为都通过自然语言的形式被详细记录下来。生成式智能体的概念为基于大语言模型的模拟仿真研究提供了重要的技术方案，并为后续在推荐系统和网络搜索等领域的应用奠定了坚实基础 [114, 310, 311]。

从上述应用案例可以看出，大语言模型为自主智能体系统带来了重要的发展机遇，未来存在着非常广阔的应用场景。为了系统性地构建基于多智能体的应用，研究人员可以基于相关的大模型智能体开源库（如 AgentScope [312]、RecAgent [114] 等）进行相关应用的开发，充分利用已有框架所实现的功能模块完成系统需求，从而提升整体的研发效率。

11.2.5 待解决的关键技术问题

尽管大语言模型智能体已经取得了重要的进展，但是它们在实际应用中仍然面临着一系列技术挑战。下面针对一些代表性的技术挑战进行介绍。

- 智能体系统的计算资源消耗. 随着大语言模型的规模不断扩展, 其在训练和部署过程中对于计算资源的消耗急剧增加。对于单个智能体来说, 通常每次动作行为都需要对大语言模型进行调用, 导致整个过程中产生了较高的调用成本。进一步, 在多智能体系统中, 当需要多个大语言模型智能体协同工作时, 资源消耗问题更为严重, 导致当前的多智能体系统往往不能扩展到较大规模智能体数量。效率问题已经成为制约其在智能体系统广泛部署的一个重要因素。因此, 研究如何优化大语言模型智能体系统的资源效率, 是当前的一個关键挑战。

- 复杂工具使用. 与人类相似, 智能体系统往往需要引入外部工具来实现特定功能, 例如使用搜索引擎从网络检索信息等。学会使用合适的工具对于拓展智能体的能力范围非常重要, 然而, 大语言模型智能体在工具使用上仍然面临着挑战。智能体常常需要应对复杂多变的环境, 为了支持其进行复杂的决策过程, 工具与大语言模型智能体之间的紧密适配显得尤为关键。然而, 现有工具的开发过程通常没有充分考虑与大语言模型的适配, 难以在复杂环境中为大语言模型提供最适合的功能支持。此外, 随着可使用工具规模的扩大, 大语言模型智能体对于新工具的可扩展性也需要进一步加强, 从而能够利用更广泛的工具解决复杂任务。

- 高效的多智能体交互机制. 在多智能体系统中, 随着智能体数量的不断增加, 智能体之间的协调和交互变得非常复杂。为了让智能体之间有效地协同工作, 需要设计高效的通信与协调机制, 以确保单个智能体能够及时准确地获取所需信息, 并做出合理的行为决策, 从而完成预期的角色与作用。目前, 开发适用于大规模智能体系统的通信协议和组织架构仍然是一个技术挑战, 需要考虑智能体的异构性、系统的可扩展性和交互的实时性等多个因素。

- 面向智能体系统的模型适配方法. 虽然大语言模型已经展现出了较强的模型能力, 但是在支撑智能体系统的基础能力方面仍然存在着一定的局限和不足。例如, 在理解复杂指令、处理长期记忆信息等方面, 现有的大语言模型的表现还需要进一步优化与改进。此外, 在构建智能体系统时, 大语言模型在维持智能体身份与行为的一致性上存在不足, 为真实模拟目标角色带来了挑战。总体来说, 需要对于大语言模型进行针对性的优化与适配, 使之更好地支撑智能体系统的有效运行。

- 面向真实世界的智能体模拟. 大语言模型智能体在虚拟仿真任务中已经取得了重要进展, 但是在真实世界环境中的应用仍面临很大挑战。首先, 现有的大语言模型智能体研究通常设置在虚拟环境中进行, 然而真实世界更加复杂, 与虚拟

环境存在着较大差异。例如，将智能体应用在机器人上时，机器人的硬件在很多时候并不能准确地工作（如机械臂操作的准确性，外界感知硬件的精度）。进一步，真实世界中所包含的信息量会远超于虚拟环境，也为大语言模型智能体有限的信息处理能力带来了挑战。此外，真实世界对于一些严重错误的容忍性远低于虚拟世界（如自动驾驶故障、种族歧视等问题）。因此，智能体在真实世界中的行为需要严格遵守人类世界的规范和标准，以确保其决策和行为的安全性。

随着大语言模型技术的不断发展，相信这些技术挑战将逐步得到解决，大语言模型智能体以及基于其的仿真系统在未来将会得到更多的应用。